

Exp: 28 Counting Semaphore in SystemC

Program:

```
// Code your testbench here.

// Uncomment the next line for SystemC modules.

// #include <systemc>

#include <systemc.h>

SC_MODULE(counting_semaphore) {

    int sem;

    void task() {

        while(true) {

            wait(1, SC_SEC);

            if(sem > 0) {

                sem--;

                cout << "Task Accessing Resource at "

                << sc_time_stamp()

                << " Remaining: " << sem << endl;

                wait(2, SC_SEC);

                sem++;

            }

        }

    }

    SC_CTOR(counting_semaphore) {

        sem = 2;

        SC_THREAD(task);

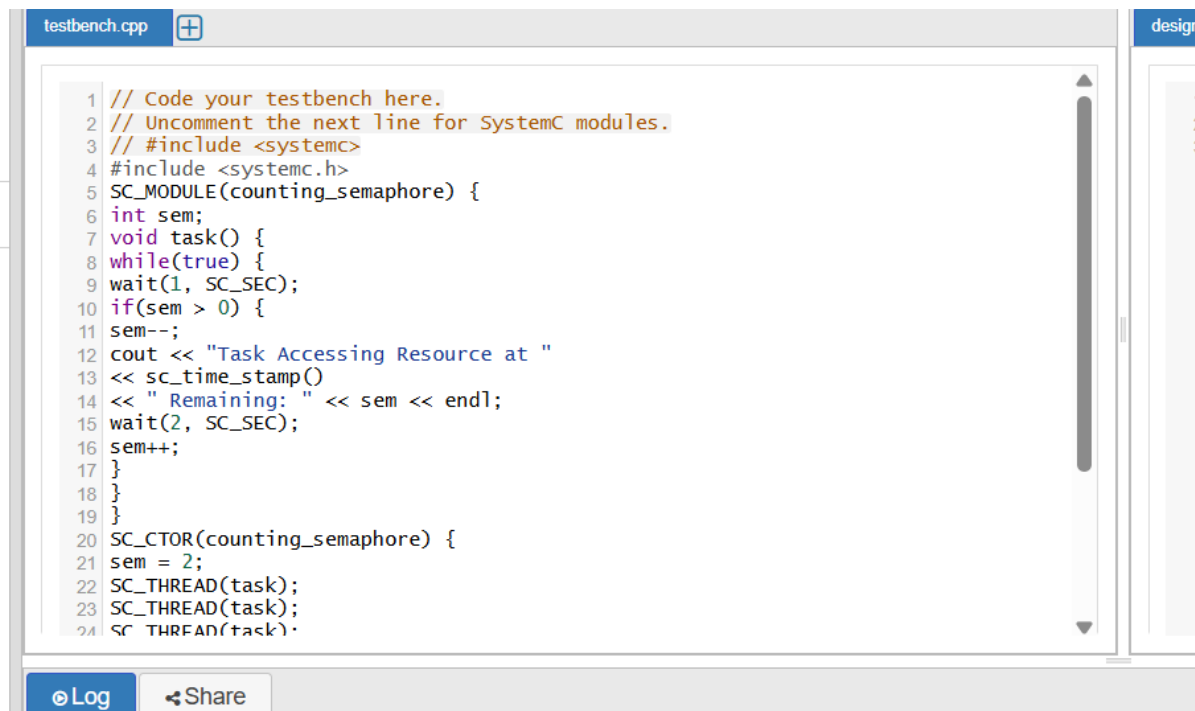
        SC_THREAD(task);

        SC_THREAD(task);

    }

};
```

```
int sc_main(int argc, char* argv[]) {  
  
    counting_semaphore obj("Counting");  
  
    sc_start(10, SC_SEC);  
  
    return 0;  
  
}
```



The screenshot shows a code editor window titled "testbench.cpp" with a plus icon in the title bar. The code is a SystemC testbench for a counting semaphore. It includes comments and standard SystemC headers. The main function defines a semaphore object, starts the simulation for 10 seconds, and returns 0. The semaphore module defines a task function that prints the task's access to the resource and its remaining count. The main function also defines the semaphore's constructor and registers the task as a thread.

```
1 // Code your testbench here.  
2 // Uncomment the next line for SystemC modules.  
3 // #include <systemc>  
4 #include <systemc.h>  
5 SC_MODULE(counting_semaphore) {  
6     int sem;  
7     void task() {  
8         while(true) {  
9             wait(1, SC_SEC);  
10            if(sem > 0) {  
11                sem--;  
12                cout << "Task Accessing Resource at "  
13                << sc_time_stamp()  
14                << " Remaining: " << sem << endl;  
15                wait(2, SC_SEC);  
16                sem++;  
17            }  
18        }  
19    }  
20    SC_CTOR(counting_semaphore) {  
21        sem = 2;  
22        SC_THREAD(task);  
23        SC_THREAD(task);  
24        SC_THREAD(task);  
25    }
```

At the bottom of the editor, there are two buttons: "Log" and "Share".